

Assignment 2 Report TT3L_Group1

Name: Yeb Woei Lun (243UC245MN)

Name: Bianca Lau Ying Xuan (242UC2426R)

Question 21: The 32 bits value stored in 0x2000 till 0x2000 + 36 is actually the date in BCD format. For example, DDMMYYYY, for example 0x01021972 is represented as 1st February 1972. If the date format is valid, store “1111” in 0x2100 till (0x2100 + 36). Else store “2222” in 0x2100 till (0x2100 + 36)

Part A: Storing initial values to address 0x2000 until 0x2024

Each group of instructions combines several immediate values using mov and add to form a 32-bit value that mimics a BCD-encoded date (format 0xDDMMYYYY), then stores it into memory start at address 0x2000 till 0x2000 + 36

```
; move r0,#0x11111111
```

```
mov r0,#0x11000000
```

```
r0 = 0001 0001 0000 0000 0000 0000 0000 0000
```

```
mov r1,#0x00110000
```

```
r1 = 0000 0000 0001 0001 0000 0000 0000 0000
```

```
mov r2,#0x00001100
```

```
r2 = 0000 0000 0000 0000 0001 0001 0000 0000
```

```
mov r3,#0x00000011
```

```
r3 = 0000 0000 0000 0000 0000 0000 0001 0001
```

```
add r0, r0, r1
```

```
r0 = r0 + r1
```

```
= 0001 0001 0000 0000 0000 0000 0000 0000
```

```
+ 0000 0000 0001 0001 0000 0000 0000 0000
```

```
= 0001 0001 0001 0001 0000 0000 0000 0000
```

```
add r0, r0, r2
```

```
r0 = r0 + r2
```

```
= 0001 0001 0001 0001 0000 0000 0000 0000
```

```
+ 0000 0000 0000 0000 0001 0001 0000 0000
```

= 0001 0001 0001 0001 0001 0001 0000 0000

add r0, r0, r3

r0 = r0 + r3

= 0001 0001 0001 0001 0001 0001 0000 0000

+ 0000 0000 0000 0000 0000 0000 0001 0001

= 0001 0001 0001 0001 0001 0001 0001 0001

= 0x11111111

mov r8, #0x2000

r8 = 0x2000

str r0, [r8] ; Store value in r0 (0x11111111) into memory at address 0x2000

; move r0, #0x22223333

mov r0, #0x22000000

r0 = 0010 0010 0000 0000 0000 0000 0000 0000

mov r1, #0x00220000

r1 = 0000 0000 0010 0010 0000 0000 0000 0000

mov r2, #0x00003300

r2 = 0000 0000 0000 0000 0011 0011 0000 0000

mov r3, #0x00000033

r3 = 0000 0000 0000 0000 0000 0000 0011 0011

add r0, r0, r1

r0 = r0 + r1

= 0010 0010 0000 0000 0000 0000 0000 0000

+ 0000 0000 0010 0010 0000 0000 0000 0000

= 0010 0010 0010 0010 0000 0000 0000 0000

add r0, r0, r2

r0 = r0 + r2

= 0010 0010 0010 0010 0000 0000 0000 0000

+ 0000 0000 0000 0000 0011 0011 0000 0000

= 0010 0010 0010 0010 0011 0011 0000 0000

add r0, r0, r3

r0 = r0 + r3

= 0010 0010 0010 0010 0011 0011 0000 0000

+ 0000 0000 0000 0000 0000 0000 0011 0011
 = 0010 0010 0010 0010 0011 0011 0011 0011
 = 0x22223333

str r0, [r8,#4] ;Store value in r0 (0x22223333) into memory at address 0x2004

; move r0,#0x31111111

mov r0,#0x31000000

r0 = 0011 0001 0000 0000 0000 0000 0000 0000

mov r1,#0x00110000

r1 = 0000 0000 0001 0001 0000 0000 0000 0000

mov r2,#0x00001100

r2 = 0000 0000 0000 0000 0001 0001 0000 0000

mov r3,#0x00000011

r3 = 0000 0000 0000 0000 0000 0000 0001 0001

add r0, r0, r1

r0 = r0 + r1

= 0011 0001 0000 0000 0000 0000 0000 0000
 + 0000 0000 0001 0001 0000 0000 0000 0000
 = 0011 0001 0001 0001 0000 0000 0000 0000

add r0, r0, r2

r0 = r0 + r2

= 0011 0001 0001 0001 0000 0000 0000 0000
 + 0000 0000 0000 0000 0001 0001 0000 0000
 = 0011 0001 0001 0001 0001 0001 0000 0000

add r0, r0, r3

r0 = r0 + r3

= 0011 0001 0001 0001 0001 0001 0000 0000
 + 0000 0000 0000 0000 0000 0000 0001 0001
 = 0011 0001 0001 0001 0001 0001 0001 0001
 = 0x31111111

str r0, [r8,#8] ;Store value in r0 (0x31111111) into memory at address 0x2008

; move r0,#0x42223333

mov r0,#0x42000000

r0 = 0100 0010 0000 0000 0000 0000 0000 0000

mov r1,#0x00220000

r1 = 0000 0000 0010 0010 0000 0000 0000 0000

mov r2,#0x00003300

r2 = 0000 0000 0000 0000 0011 0011 0000 0000

mov r3,#0x00000033

r3 = 0000 0000 0000 0000 0000 0000 0011 0011

add r0, r0, r1

r0 = r0 + r1

= 0100 0010 0000 0000 0000 0000 0000 0000
+ 0000 0000 0010 0010 0000 0000 0000 0000
= 0100 0010 0010 0010 0000 0000 0000 0000

add r0, r0, r2

r0 = r0 + r2

= 0100 0010 0010 0010 0000 0000 0000 0000
+ 0000 0000 0000 0000 0011 0011 0000 0000
= 0100 0010 0010 0010 0011 0011 0000 0000

add r0, r0, r3

r0 = r0 + r3

= 0100 0010 0010 0010 0011 0011 0000 0000
+ 0000 0000 0000 0000 0000 0000 0011 0011
= 0100 0010 0010 0010 0011 0011 0011 0011
= 0x42223333

str r0, [r8,#12] ;Store value in r0 (0x42223333) into memory at address 0x200C

; move r0,#0x51111111

mov r0,#0x51000000

r0 = 0101 0001 0000 0000 0000 0000 0000 0000

mov r1,#0x00110000

r1 = 0000 0000 0001 0001 0000 0000 0000 0000

mov r2,#0x00001100

r2 = 0000 0000 0000 0000 0001 0001 0000 0000

mov r3,#0x00000011

r3 = 0000 0000 0000 0000 0000 0000 0001 0001

add r0, r0, r1

$r0 = r0 + r1$
= 0101 0001 0000 0000 0000 0000 0000 0000
+ 0000 0000 0001 0001 0000 0000 0000 0000
= 0101 0001 0001 0001 0000 0000 0000 0000

add r0, r0, r2

$r0 = r0 + r2$
= 0101 0001 0001 0001 0000 0000 0000 0000
+ 0000 0000 0000 0000 0001 0001 0000 0000
= 0101 0001 0001 0001 0001 0001 0000 0000

add r0, r0, r3

$r0 = r0 + r3$
= 0101 0001 0001 0001 0001 0001 0000 0000
+ 0000 0000 0000 0000 0000 0000 0001 0001
= 0101 0001 0001 0001 0001 0001 0001 0001
= 0x51111111

str r0, [r8,#16] ;Store value in r0 (0x51111111) into memory at address 0x2010

; move r0,#0x62223333

mov r0,#0x62000000

$r0 = 0110\ 0010\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000$

mov r1,#0x00220000

$r1 = 0000\ 0000\ 0010\ 0010\ 0000\ 0000\ 0000\ 0000$

mov r2,#0x00003300

$r2 = 0000\ 0000\ 0000\ 0000\ 0011\ 0011\ 0000\ 0000$

mov r3,#0x00000033

$r3 = 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0011\ 0011$

add r0, r0, r1

$r0 = r0 + r1$
= 0110 0010 0000 0000 0000 0000 0000 0000
+ 0000 0000 0010 0010 0000 0000 0000 0000
= 0110 0010 0010 0010 0000 0000 0000 0000

add r0, r0, r2

$r0 = r0 + r2$

= 0110 0010 0010 0010 0000 0000 0000 0000
 + 0000 0000 0000 0000 0011 0011 0000 0000
 = 0110 0010 0010 0010 0011 0011 0000 0000

add r0, r0, r3

r0 = r0 + r3

= 0110 0010 0010 0010 0011 0011 0000 0000
 + 0000 0000 0000 0000 0000 0000 0011 0011
 = 0110 0010 0010 0010 0011 0011 0011 0011
 = 0x62223333

str r0, [r8,#20] ;Store value in r0 (0x62223333) into memory at address 0x2014

; move r0,#0x71111111

mov r0,#0x71000000

r0 = 0111 0001 0000 0000 0000 0000 0000 0000

mov r1,#0x00110000

r1 = 0000 0000 0001 0001 0000 0000 0000 0000

mov r2,#0x00001100

r2 = 0000 0000 0000 0000 0001 0001 0000 0000

mov r3,#0x00000011

r3 = 0000 0000 0000 0000 0000 0000 0001 0001

add r0, r0, r1

r0 = r0 + r1

= 0111 0001 0000 0000 0000 0000 0000 0000
 + 0000 0000 0001 0001 0000 0000 0000 0000
 = 0111 0001 0001 0001 0000 0000 0000 0000

add r0, r0, r2

r0 = r0 + r2

= 0111 0001 0001 0001 0000 0000 0000 0000
 + 0000 0000 0000 0000 0001 0001 0000 0000
 = 0111 0001 0001 0001 0001 0001 0000 0000

add r0, r0, r3

r0 = r0 + r3

= 0111 0001 0001 0001 0001 0001 0000 0000
 + 0000 0000 0000 0000 0000 0000 0001 0001

= 0111 0001 0001 0001 0001 0001 0001 0001

= 0x71111111

str r0, [r8,#24] ;Store value in r0 (0x71111111) into memory at address 0x2018

; move r0,#0x82223333

mov r0,#0x82000000

r0 = 1000 0010 0000 0000 0000 0000 0000 0000

mov r1,#0x00220000

r1 = 0000 0000 0010 0010 0000 0000 0000 0000

mov r2,#0x00003300

r2 = 0000 0000 0000 0000 0011 0011 0000 0000

mov r3,#0x00000033

r3 = 0000 0000 0000 0000 0000 0000 0011 0011

add r0, r0, r1

r0 = r0 + r1

= 1000 0010 0000 0000 0000 0000 0000 0000

+ 0000 0000 0010 0010 0000 0000 0000 0000

= 1000 0010 0010 0010 0000 0000 0000 0000

add r0, r0, r2

r0 = r0 + r2

= 1000 0010 0010 0010 0000 0000 0000 0000

+ 0000 0000 0000 0000 0011 0011 0000 0000

= 1000 0010 0010 0010 0011 0011 0000 0000

add r0, r0, r3

r0 = r0 + r3

= 1000 0010 0010 0010 0011 0011 0000 0000

+ 0000 0000 0000 0000 0000 0000 0011 0011

= 1000 0010 0010 0010 0011 0011 0011 0011

= 0x82223333

str r0, [r8,#28] ;Store value in r0 (0x82223333) into memory at address 0x201C

; move r0,#0x91111111

mov r0,#0x91000000

r0 = 1001 0001 0000 0000 0000 0000 0000 0000

mov r1,#0x00110000

r1 = 0000 0000 0001 0001 0000 0000 0000 0000

mov r2,#0x00001100

r2 = 0000 0000 0000 0000 0001 0001 0000 0000

mov r3,#0x00000011

r3 = 0000 0000 0000 0000 0000 0000 0001 0001

add r0, r0, r1

r0 = r0 + r1

= 1001 0001 0000 0000 0000 0000 0000 0000
+ 0000 0000 0001 0001 0000 0000 0000 0000
= 1001 0001 0001 0001 0000 0000 0000 0000

add r0, r0, r2

r0 = r0 + r2

= 1001 0001 0001 0001 0000 0000 0000 0000
+ 0000 0000 0000 0000 0001 0001 0000 0000
= 1001 0001 0001 0001 0001 0001 0000 0000

add r0, r0, r3

r0 = r0 + r3

= 1001 0001 0001 0001 0001 0001 0000 0000
+ 0000 0000 0000 0000 0000 0000 0001 0001
= 1001 0001 0001 0001 0001 0001 0001 0001
= 0x91111111

str r0, [r8,#32] ;Store value in r0 (0x91111111) into memory at address 0x2020

; move r0,#0xA2223333

mov r0,#0xA2000000

r0 = 1010 0010 0000 0000 0000 0000 0000 0000

mov r1,#0x00220000

r1 = 0000 0000 0010 0010 0000 0000 0000 0000

mov r2,#0x00003300

r2 = 0000 0000 0000 0000 0011 0011 0000 0000

mov r3,#0x00000033

r3 = 0000 0000 0000 0000 0000 0000 0011 0011

add r0, r0, r1

$r0 = r0 + r1$

= 1010 0010 0000 0000 0000 0000 0000 0000
+ 0000 0000 0010 0010 0000 0000 0000 0000
= 1010 0010 0010 0010 0000 0000 0000 0000

add r0, r0, r2

$r0 = r0 + r2$

= 1010 0010 0010 0010 0000 0000 0000 0000
+ 0000 0000 0000 0000 0011 0011 0000 0000
= 1010 0010 0010 0010 0011 0011 0000 0000

add r0, r0, r3

$r0 = r0 + r3$

= 1010 0010 0010 0010 0011 0011 0000 0000
+ 0000 0000 0000 0000 0000 0000 0011 0011
= 1010 0010 0010 0010 0011 0011 0011 0011
= 0xA2223333

str r0, [r8,#36] ;Store value in r0 (0xA2223333) into memory at address 0x2024

Part B: Function to check BCD valid and date valid

BCD valid is ensure each 4-bit nibble should be 0x0 to 0x9.

- Load the 32-bit value from memory into R0.
- Loop 8 times (each 32-bit value has 8 nibbles).
- Shift R0 right by R3 (starts at 28), get one nibble.
- AND with 0xF to keep only 4 bits.
- If nibble $\geq 0xA$, set to 1 (means invalid).
- Otherwise, keep it.
- Add shifted nibble back into R1.
- After 8 loops, R1 has the cleaned (or unchanged) version.
- Store R1 to memory.

MOV R8, #0x2000 ;Reset memory place for later to store

MOV R10, #10 ;10 iteration for 10 value

loop

LDR R0, [R8] ;take the value to check, Exp: R0 =
#0x11111111

MOV R1, #0 ;reset r1

MOV R2, #8 ;8 numbers to compare (4 bits $\times$ 8 = 32 bits).

MOV R3, #28 ;32bit - 4 bit(byte) = 28 (the fuction will check
number 1 by 1, each time use 4 bit)

check

MOV R4, R0, LSR R3 ;masking left 4 bit, shift
R0(#0x11111111) right by R3(#28) bits, so the nibble we want to check is now at the lowest 4 bits.

R0 = 0x11111111 = 0001 0001 0001 0001 0001 0001 0001 0001

Shift right 28 bits = 0000 0000 0000 0000 0000 0000 0000 0001

R4 = 0x1

AND R4, R4, #0xF ;remain the 4 bit
R4= 0000 0000 0000 0000 0000 0000 0000 0001
 AND 0000 0000 0000 0000 0000 0000 0000 1111

= 0000 0000 0000 0000 0000 0000 0000 0001

= 0x1

CMP **R4, #0xA** ;compare the value whether within 0-9

BLT **valid** ;if less then (within)

R4=0x1 < 0xA , is valid BCD, skip to valid

MOV **R4, #1** ;if excess become 1 (Only executed if R4 ≥ 0xA)

valid

ADD **R1, R1, R4, LSL R3** ;put back the value

After 1st loop: R1 = 0, R4 = 0x1 = 0001, R3 = 28

R1 = R1 + R4 left shift R3

= 0 + 0x1 left shift 28

= 0x10000000

SUB **R3, R3, #4** ;update for next number

R3 = R3 - 4

= 28-4

= 24

SUB **R2, R2, #1** ;Indicate 1 number done

CMP **R2, #0** ;check if all number already compared

BNE **check** ; If more nibbles to check, loop again

Loop Repeats 8 Times, each time, it extracts a nibble with LSR R3, AND #0xF to checks if it's < 0xA, then shifts and adds into R1. Because all 8 nibbles in 0x11111111 are 0x1, all are valid, and get added into R1 at the correct position.

STR **R1, [R8], #4** ;store value and add 4 to compare next

value

R1 = 0x11111111, it unchanged, because all digits are valid BCD.

SUB **R10, R10, #1** ; Decrement the loop counter by 1 to

indicate one iteration is completed

CMP **R10, #0** ; Compare the updated value to check if all

iterations are done

BNE **loop** ; If is not equal to 0, branch to the label loop to

repeat the process

Part C: Check date validation

Step 1: Date loop

- Load 32-bit value from memory (0x2000, 0x2004, etc.).
- Extract day (DD) and month (MM):
 - $R2 = (R1 \gg 16) \& 0xFF \rightarrow \text{month}$
 - $R3 = (R1 \gg 24) \& 0xFF \rightarrow \text{day}$

Step 2: Check Month Validity

- Month must be: $1 \leq MM \leq 12$
- If not: jump to invalid_date

Step 3: Check Day Validity

- Day must be: $1 \leq DD \leq 31$
- If not: jump to invalid_date

Step 4: February Check (With Leap Year Logic)

- If month == 2:
 1. Extract year: $R6 = R1 \& 0xFFFF$
 2. Convert from BCD to Decimal:
 - Use logic to get thousands, hundreds, tens, and units:
 - $\text{LSR } \#12 \rightarrow \text{thousands}$
 - $\text{LSR } \#8 \rightarrow \text{hundreds}$
 - $\text{LSR } \#4 \rightarrow \text{tens}$
 - $\text{AND } \#0xF \rightarrow \text{units}$
 - Loop to compute total value into R11
 - Final decimal year stored in R11
 3. Determine leap year:
 - If $R11 \% 400 == 0 \rightarrow \text{leap year}$
 - Else if $R11 \% 100 == 0 \rightarrow \text{not leap}$
 - Else if $R11 \% 4 == 0 \rightarrow \text{leap}$
 - Else $\rightarrow \text{not leap}$

4. Then:

- If leap year → max day = 29
- Else → max day = 28
- If DD > max → jump to invalid_date

Step 5: 30-Day Months Check

- For months with only 30 days (April, June, Sept, Nov):
 - If DD > 30, jump to invalid_date

Step 6: Valid or Invalid Flag

- If valid: R5 = 0x1111 → store this
- If invalid: R5 = 0x2222
- Store into memory from 0x2100

are stored **MOV** **R8, #0x2000** ; set memory place where the BCD values

validation results **MOV** **R4, #0x2100** ; set memory place where will store

MOV **R10, #10** ;10 iteration for 10 value
date_loop

LDR **R1, [R8]** ; get memory address 2000 value
R8 = 0x2000 and memory[0x2000] = 0x11111111
R1 = memory[R8] = 0x11111111

; Function check month valid or not

MOV **R2, R1, LSR #16** ;masking to get 8th to 5th place bit
that represent month

R1= 0x11111111 = 0001 0001 0001 0001 0001 0001 0001 0001

R2 = R1 LSR 16
= 0000 0000 0000 0000 0001 0001 0001 0001
= 0x1111

AND **R2, R2, #0xFF** ; mask the lower 8 bits to isolate the
Month byte
R2 = R2 AND #0xFF

```

=      0000 0000 0000 0000 0001 0001 0001 0001
AND 0000 0000 0000 0000 0000 0000 1111 1111
= 0000 0000 0000 0000 0000 0000 0001 0001
= 0x11

```

```

CMP      R2, #0x1 ;check whether less than 1
BLT      invalid_date ; if invalid, skip to invalid_date
CMP      R2, #0x12 ;check whether greater than 12
BGT      invalid_date ; if invalid, skip to invalid_date

```

(if all valid (between 1 to 12) , continue check day valid)
0x1 < 0x11 < 0x12, this date is valid and continue check day valid

; Function check day valid or not

```

MOV      R3, R1, LSR #24 ;masking to get 8th and 7th place bit
that represent day (24 = 6 * 4 byte)
R1 = 0x11111111 = 0001 0001 0001 0001 0001 0001 0001 0001
R3 = R1 shift right 24 = 0000 0000 0000 0000 0000 0000 0001 0001 = 0x11

```

```

AND      R3, R3, #0xFF ;get only date
R3 = R3 AND #0xFF
=      0000 0000 0000 0000 0000 0000 0001 0001
AND 0000 0000 0000 0000 0000 0000 1111 1111
= 0000 0000 0000 0000 0000 0000 0001 0001
= 0x11

```

```

CMP      R3, #0x1 ;check whether less than 1
BLT      invalid_date ; if invalid, skip to invalid_date

CMP      R3, #0x31 ;check whether greater than 31
BGT      invalid_date ; if invalid, skip to invalid_date

```

; February check leap year (February check)

```

CMP      R2, #0x2 ;check month = 2 (February)
BEQ      February ; skip to February check

```

; month that only got 30 days (April, June, September, November)

```

CMP      R2, #0x4 ;check month = 4 (April)
BEQ      check_30days ; skip to check_30days

```

CMP	R2, #0x6 ;check month = 6 (June)
BEQ	check_30days ; skip to check_30days
CMP	R2, #0x9 ;check month = 9 (September)
BEQ	check_30days ; skip to check_30days
CMP	R2, #0x11 ;check month = 11 (November)
BEQ	check_30days ; skip to check_30days

back

MOV R5, #0x1100 ; if all valid = 1111 (Immediate must be creatable by rotating an 8-bit number right within a 32-bit word)

ADD R5, R5, #0x11

$R5 = R5 + 0x11$
 $= 0x1100 + 0x11$
 $= 0x1111$

B store_result ; skip to store_result

invalid_date

MOV R7, #0x22 ;if invalid = 2222 (Immediate must be creatable by rotating an 8-bit number right within a 32-bit word)

MOV R9, #0x2200

ADD R5, R7, R9

$R5 = R7 + R9$
 $= 0x22 + 0x2200$
 $= 0x2222$

B store_result ; skip to store_result

check_30days

CMP R3, #0x30 ;check whether greater than 30

BGT invalid_date ;if invalid, skip to invalid_date

B back ;skip to back

February

MOV R6, R1 ;move year to R1

Example: R1=0x29021998

R6 = R1 =0x29021998= 0010 1001 0000 0010 0001 1001 1001 1000

MOV R12, #0xFF

R12 = 0xFF = 0000 0000 0000 0000 0000 0000 1111 1111

ADD R12,R12, #0xFF00

R12 = 0000 0000 0000 0000 0000 0000 1111 1111
 + 0000 0000 0000 0000 1111 1111 0000 0000
 = 0000 0000 0000 0000 1111 1111 1111 1111
 = 0xFFFF

AND R6, R6, R12 ;remain 4bit

R6 = R6 AND R12
 = 0010 1001 0000 0010 0001 1001 1001 1000
 AND 0000 0000 0000 0000 1111 1111 1111 1111
 = 0000 0000 0000 0000 0000 0001 1001 1001 1000
 = 0x1998

MOV R7, R6 ;copy of R6

R7 = R6 = 0x1998

MOV R11, #0 ; Reset R11 for convert hex to decimal

MOV R12, #0 ; digit counter

; Covert hexa to decimal for the modular to get whether leap year or

not

MOV R7, R6, LSR #12 ;get thousands place

R7 = R6 LSR 12
 = 0x1

AND R7, R7, #0xF ;remain only 1 bit

R7 = R7 AND 0xF
 = 0x1

CMP R7, #0 ;if 0 means no need to add decimal

BEQ skip_1000 ; if thousands place > 0, minus thousands
place and add decimal 1000

loop_1000

ADD R11, R11, #1000 ; + 1000 decimal

R11 = R11 + 1000
 = 0 + 1000
 = 1000

SUBS R7, R7, #1 ; - thousands place

R7 = R7 -1 = 1 - 1 = 0

thousands digit : 2	BNE	loop_1000 ;if still got thousands place then repeat
skip_1000		
R6 = 0x1998 R7 = 0x19	MOV	R7, R6, LSR #8 ;get hundreds place
R7 = 0x9 hundreds digit : 9	AND	R7, R7, #0xF ;remain only 1 bit
	CMP	R7, #0 ;if 0 means no need to add decimal
and add decimal 100	BEQ	skip_100 ; if hundreds place > 0, minus hundreds place
loop_100		
R11=1000+900 =1900	ADD	R11, R11, #100 ; + 100 decimal ; loop 9 times
	SUBS	R7, R7, #1 ; - hundreds place
	BNE	loop_100 ;if still got hundreds place then repeat
skip_100		
R7 = 0x199	MOV	R7, R6, LSR #4 ;get tens place (same logic with above)
R7 = 0x9 tens digit : 9	AND	R7, R7, #0xF
	CMP	R7, #0 ; if 0 means no need to add decimal
add decimal 10	BEQ	skip_10 ; if tens place > 0, minus hundreds place and
loop_10		
; loop 9 times, R11 = 1900+90 R11 = 1990	ADD	R11, R11, #10
R7 = 2 - 1 =1	SUBS	R7, R7, #1
	BNE	loop_10 ; if still got tens place then repeat
skip_10		

AND **R7, R6, #0xF** ;get ones place
 R6=0x1998, R7= 0x8
CMP **R7, #0** ; if 0 means no need to add decimal
BEQ **skip_1** ; if ones place = 0 then skip

loop_1

ADD **R11, R11, #1**
 ;loop 8 times
 R11 = 1990 +8 = 1998
SUBS **R7, R7, #1**
 R7 = R7 + 1
BNE **loop_1** ; if still got tens place then repeat

skip_1

;Check whether Leap year or not (Leap years occur when the year is
 divisible by 4, except for years divisible by 100, unless they are also divisible by 400)

; modular 400

MOV **R7, R11** ;R1 is the value after convert to decimal
 R7 = 1998

MOV **R9, #400**
 R9 =400

B **mod400** ; skip to mod400

mod400_back

CMP **R11, #0** ;can be divide by 400 so it is leap year
BEQ **leap_year** ;if false continue to mod 100

;=====

 modular 100 =====

MOV **R7, R11** ;R11 now is value that already modular 400
 R7 =1998

MOV **R9, #100**
 R9 =100

B **mod100** ; skip tp mod100

mod100_back

CMP **R11, #0** ;can be divide by 100 so it is not a leap year
BEQ **not_leap_year** ;if false continue to mod4

;
modular 4

MOV **R7, R11** ;R11 now after modular 100

R7 =2024

MOV **R9, #4**

R9 =4

B **mod4** ; skip to mod4

mod4_back

CMP **R11, #0** ;can be divide by 4 so it is leap year
BEQ **leap_year**
;else not leap year
B **not_leap_year** ; skip to not_leap_year

mod400

MOV **R11, R7**

mod400_loop

CMP **R11, R9** ;R9 = 400
BLO **mod400_back** ;R11 < 400 then back
SUB **R11, R11, R9** ;minus 400 and store back

R11 = 1998 = 400 = 1598

B **mod400_loop** ; Continue until R11 < 400 → will
become 398, R11 = 1998 % 400 = 398

mod100

MOV **R11, R7** ;put back the value after mod 400

R11=1998

mod100_loop

CMP **R11, R9** ;R11 < 100 then back

BLO **mod100_back**

SUB **R11, R11, R9** ;minus 100 and store back

R11 = 1998 – 100 repeatly until R11=98

B **mod100_loop**

mod4

MOV **R11, R7** ;put back the value after mod 100

mod4_loop

CMP **R11, R9** ;R11 < 4 then back

BLO **mod4_back**

SUB **R11, R11, R9** ;minus 4 and store back

R11 = 1998 -4 repeatly until R11 = 2

B **mod4_loop**

not_leap_year

February **CMP** **R3, #0x28** ;not leap year so only got 28 days in

BGT **invalid_date**

B **back**

leap_year

CMP **R3, #0x29** ;leap year so got 29 days in February

BGT **invalid_date**

B **back**

store_result

STR **R5, [R4], #4** ;R5 = result, add result to address 2100
and add 4 then repeat

ADD **R8, R8, #4** ;R8 = address 2000, add 4 for the next value

SUB **R10, R10, #1** ;iteration -1

```
CMP      R10,#0 ;when iteration == 0
BNE      date_loop ;if not repeat

END
```

Result:

View Memory Contents

Start address: 0x2000 End address: 0x3000

Word Address	Byte 3	Byte 2	Byte 1	Byte 0	Word Value
0x2000	0x11	0x11	0x11	0x11	0x11111111
0x2004	0x22	0x22	0x33	0x33	0x22223333
0x2008	0x31	0x11	0x11	0x11	0x31111111
0x200C	0x42	0x22	0x33	0x33	0x42223333
0x2010	0x51	0x11	0x11	0x11	0x51111111
0x2014	0x62	0x22	0x33	0x33	0x62223333
0x2018	0x71	0x11	0x11	0x11	0x71111111
0x201C	0x82	0x22	0x33	0x33	0x82223333
0x2020	0x91	0x11	0x11	0x11	0x91111111

Word Value Format ☒ Dec ☐ Hex

Memory Map Key ☒ Instructions ☐ Data

View Memory Contents

Start address: 0x2000 End address: 0x3000

Word Address	Byte 3	Byte 2	Byte 1	Byte 0	Word Value
0x2020	0x91	0x11	0x11	0x11	0x91111111
0x2024	0x12	0x22	0x33	0x33	0x12223333
0x2028	0x0	0x0	0x0	0x0	0x0
0x2100	0x0	0x0	0x11	0x11	0x1111
0x2104	0x0	0x0	0x22	0x22	0x2222
0x2108	0x0	0x0	0x22	0x22	0x2222
0x210C	0x0	0x0	0x22	0x22	0x2222
0x2110	0x0	0x0	0x22	0x22	0x2222
0x2114	0x0	0x0	0x22	0x22	0x2222

Word Value Format ☒ Dec ☐ Hex

Memory Map Key ☒ Instructions ☐ Data

View Memory Contents

Start address: 0x2000 End address: 0x3000

Word Address	Byte 3	Byte 2	Byte 1	Byte 0	Word Value
0x2108	0x0	0x0	0x22	0x22	0x2222
0x210C	0x0	0x0	0x22	0x22	0x2222
0x2110	0x0	0x0	0x22	0x22	0x2222
0x2114	0x0	0x0	0x22	0x22	0x2222
0x2118	0x0	0x0	0x22	0x22	0x2222
0x211C	0x0	0x0	0x22	0x22	0x2222
0x2120	0x0	0x0	0x22	0x22	0x2222
0x2124	0x0	0x0	0x22	0x22	0x2222
0x2128	0x0	0x0	0x0	0x0	0x0

Word Value Format ☒ Dec ☐ Hex

Memory Map Key ☒ Instructions ☐ Data